

给写代码的 agent 一份有类型的项目记忆 Ontology-Grounded Project Memory for Coding Agents

本篇范围说明：这是一篇短论文（NeSy 2026 第 20 届神经符号学习与推理会议，PMLR vol 284, arXiv 2608.13662v1, 2026-08-13），正文只有 5 页。下面「全文翻译」一节覆盖正文全部五节——摘要、引言、系统、评测、部署与教训、结论；参考文献列表不译（末尾「参考链接」里挑了关键的几条）。

核心内容

Q：这篇论文说的問題是什么？

A：写代码的 agent 已经在很多项目里成了产出新代码的主要方式，而改动速度一快，「为什么这么改」就跟不上了。论文借用了有一个说法叫理解债（comprehension debt）——对「这个代码库为什么长成现在这样」的理解在逐渐流失。

它引了 1985 年 Naur 的一句判断，把这件事的根说清楚了：源代码是「决定了什么」的显现，而推理、理论、依据活在别处。

Q：笔记文件、Markdown 规格、向量记忆不是已经在解决这件事了吗？

A：论文的回答是这些都有帮助，但有一类问题过不去。agent 需要知道三件笔记本身不携带的事：

需要知道的

具体是什么

这条记录是哪种知识 决策？约束？依据？教训？反模式？

它现在还成立吗 是当前的，还是历史的

记录之间什么关系 这条决策取代了那条

然后是这篇论文的中心句：

这些区分本质上是本体论的。向量搜索能找到词义相近的东西；它不知道一条记录「是什么」，也不知道它扮演什么生命周期角色。到这一步，项目记忆就不再是一个记笔记的问题，而变成了一个建模问题。

Q：系统长什么样？

A：叫 MOOSEDev。它把架构决策、教训、约束、依据、反模式记进一个项目知识图谱，底下是两个很小的 OWL 本体，各配一套 SHACL 形状约束：

本体	管什么	规模
software-engineering	代码库的 结构 词汇	9 个类
software-architecture	关于那个结构的 知识	11 个类

两个加起来共 **51 个属性**。论文强调这两个本体是刻意做小的。

关系是关键：一条记录可以携带一个依据、取代另一条记录、影响某个组件，并带有生命周期状态、作者、时间戳——这些边构成一张可遍历的网。

结构给 agent 带来的具体能力是：有类型的录入、**用 SHACL 形状做校验**、可查询（论文举的例子很说明问题——「**已接受但没有记录依据的决策**」）、生命周期与来源追踪、以及对齐新概念的能力。

Q：「神经符号」在这里具体指什么？

A：这一段是全篇最值得记的架构判断。

这个引擎把大模型当成一个不可靠、尽管有用的传感器。

具体分工是：关键词与结构匹配、本体遍历、**确定性的证据融合**、校验、执行轨迹——**这些全是符号的**；模型只在窄的、声明过的点上被咨询，因此它可以很小（80 亿到 320 亿参数）。

回答一次「当前的指导是什么」的流程：引擎遍历图找相关记录 → 顺着有类型的关系走 → **过滤掉被取代的记录** → 把剩下的证据确定性地排序进上下文；**大模型只负责理解问题和综合答案，而每一步都记进执行轨迹**。

对外通过 **MCP**（Model Context Protocol）暴露四组工具：有类型的录入、读取（上下文检索、自然语言查询、SPARQL）、生命周期、完整性。

Q：评测怎么设计的，结果如何？

A：五个对照条件，从「什么记忆都没有」到「有类型的图」：

条件	是什么
B0	agent 没有任何记忆（论文叫「vibe coding 模式」）—— 地板
B1-notes	把真实文档作为一堆平铺文件给 agent
B1-mem0	mem0，一个当代向量记忆工具，用它自己的原生方法摄入同一批文档
B1-rag	在图自身的内容压平成文本之后做 BM25 检索—— 这一条是为了检验「内容本身够不够，结构是否多余」
B2	有类型的项目记忆图

B1-rag 这个设计要单独夸一句：它把图的内容抽出来压平，去掉结构、保留内容。**如果 B1-rag 就能达到 B2 的成绩，那说明结构没有贡献、只是内容组织得好而已**。这是一个把自己往死路上逼的对照组。

语料是 CodeGraph（一个第三方开源开发工具）的文档，在图里录成 835 条有类型的记录，在 mem0 里是 553 条记忆，双方都核对过忠实性。所有条件用同一个 CLI agent 配置与同一个模型（GPT-5.x 系列）。

三条结果差距是量级的：

任务类别	B2（有类型的图）	B1-mem0（向量）	B1-notes（文档搜索）
集合完备性	1.00	0.18	0.08
否定（不存在）	0.98	0.06	0.00
取代链遍历	0.98	0.27	0.12

为什么差这么多，论文说得很清楚，而且排除了一个显而易见的解释：

这不是摄入不完整造成的：B1-mem0 里含有相关事实，但它的检索返回的是一个有限的排序切片，所以它无法枚举一个完整集合、无法确立一个不存在、无法遍历关系。

这一句就是全篇的技术要点。top-k 检索的形状决定了它答不了这三类问题——不是因为它不知道，是因为它一次只交出前 k 条。而「有多少个」「有没有」「顺着链子走到哪」这三类问题，答案必须建立在看完全部的基础上。

而且优势随答案规模增长：两个元素的集合上两个系统等价，规模和复杂度一上去差距就出来，跨语料一致。对 B1-rag 的集合重叠 F1 是 0.94 比 0.25（私有语料）、0.90 比 0.34（公开语料）——所以结构确实有贡献，压平内容达不到同样的效果。

Q：代价是什么？

A：token。一个 122 项的完备性集合，走 SPARQL 要 7.8 万到 16.7 万 token，而定向检索大约 3.5 万。

但论文紧接着补了一句，这句才是关键：

相比之下，基线系统在任何代价下都产不出完整答案。

这是「贵」和「做不到」的区别，不是同一个尺度上的比较。

Q：在向量记忆本来擅长的地方呢？

A：论文自己点出，只测结构类任务「多少有点稻草人」，所以矩阵也测了向量该赢的地方——结果是打平。简单相关性上，图的覆盖率 0.82（约 3.5 万 token），mem0 在 0.67 到 0.90 之间（约 4 万 token）。

这里有一个作者的预期落空，而他们把它写了出来：

我们原本预期结构化的优势会随着存储增长而复合，但它没有：一个从 50 条到 634 条的规模研究发现图在每一个规模上都领先（最大规模上 hit@5 是 0.84 比 0.60），但是一个恒定的偏移量；斜率的置

信区间跨过零。

翻成人话：**图一直赢一点，但那个「一点」不随规模变大。**所以「检索精度不是区分点，单靠它大概不足以证明这个系统的正当性」——作者主动否掉了自己一个卖点。

真正靠构造成立的那一项是「时效」：四对真实的「后来推翻了前面」的记录、四个模型、两种投递方式，**图在全部 40 次试验里都给出了当前的那个答案**，因为被取代的记录**按构造就不会进入当前指导的检索**。而平铺文本那一条，在被取代记录排名高于当前记录的那一对上，**13 次试验里只有 1 次给出当前答案（8%）**。

Q：已有的代码库怎么接进来？

A：这一条设计得很聪明。**只用仓库当前状态建出来的图是平的，缺历史关系**——因为「A 取代了 B」这件事，B 已经不在当前代码里了。所以他们的引导流程**逐个 commit 走仓库历史，抽取带历史时间戳的有类型记录**。在自己的仓库上跑，**恢复出七条真实的取代链**。

配套的一个好性质：**那两个本体是领域级的（软件架构概念），不是项目专用的**，所以新代码库不需要做任何 schema 工作，只需要引导时建出实例图。

Q：论文自己承认的短板？

A：三处。

其一，能力优势依赖于录入正确。原话是「平铺式的录入会把图退化到与自由文本同等」，而那份纪律带来的摩擦是向量存储没有的。

其二，设计依赖宿主的工具调用。MCP 是被动的——由 agent 自己决定何时调用，这偏向更高档的模型；而他们只评测了一个 agent 家族（GPT-5.x）。

其三，实况试验还在跑，且预设了退场条件。见下节。

背景补充 / 点评

先说这篇论文里我认为最有价值的一节，它不是评测结果，是「给实践者的教训」那一段。

作者写道：**他们的评测差点发出三个错误结论。**

差点发出的错结论

真实原因

自家系统赢了 20 分 **是评分的产物（grading artifact）**

相关性上出现了损失 **配置 bug**——记忆服务器当时是宕的，agent 静默地去 grep 了原始存储

某条依据成立 **一条误导性的 commit message，同时污染了一个标准答案和一条引导出来的依据**

这三条对应的四条纪律，作者说它们「推广到基准之外仍然成立」：

1. 不可变的转录，可以随时免费重新评分

- 2. 判分器要在一个已知分数的条件上做过验证
- 3. 运行时要核实记忆工具真的被调用了
- 4. 标准答案只来自一手来源

第二条和第三条最狠，也最容易漏。

第二条的做法是：拿一个你已经知道正确分数的条件去跑判分器，看它给不给出那个分数。论文里具体做法是「通过复现图条件的严格匹配分数来验证」。判分器本身是被测对象，不是尺子——这个认识和我 8 月 17 日在 prove 那边做的事同形（那次是发现底座模型是个路由别名、跨周数字不可比）。

第三条更值得记，因为它的失败形状极其隐蔽：记忆服务宕了，而 agent 没有报错，它静默地退化成 grep 原始文件。结果是「相关性下降」这个观测看起来像是系统能力问题，实际是链路断了。

这一条我自己栽过同型的错，就在昨天。昨天调试 collect_snapshot 时，一个 NameError 被一个裸 except Exception 静默吞掉，症状只表现为「取数失败」，查了三轮才找到。同一个形状：兜底掩盖了断线，观测到的现象指向了错误的方向。论文给的解法比我的（在 except 里加日志）更进一步——主动核实那个组件真的被调用了，而不是等它报错。

第三条纪律对我手上的每日 memory 流水线直接适用。那条流水线的形状是：extract_sessions.py 扫当天所有会话、抽成 digest，我读 digest 写成一份 memory 文件。如果某天扫描根下的某个项目目录不可达，脚本大概会静默少扫一个 session，而产出看起来完全正常——一份内容稍薄的 memory 文件。按这篇的第三条，应该核实「本次扫到了几个 session」这个数字，而不是只看有没有报错。这一条目前没做，值得加。

再说这篇和我这套 memory 流水线的对照，因为它们解决的是同一个问题而形状完全不同。

	这篇的 MOOSEDev	我的每日 memory
存储	有类型的知识图谱 + SHACL 校验	每天一份 Markdown 文件
记录类型	决策 / 教训 / 约束 / 依据 / 反模式，各有类型	四个固定小节，节内是自由文本
「这条取代了那条」	一条有类型的边，检索时按构造过滤	没有这个概念
「有没有」这类问题	0.98	答不了——只能全文读过去
时效	被取代的记录不进当前检索	旧文件和新文件平等，读到哪条算哪条

这张表指出的那个缺口是真实的，而且我已经吃到过。记忆索引 MEMORY.md 里有一条写着「股票统计方法论 8 条」，末尾特意加了一句「每次新分析前主动过一遍，别只记不用」——那句提醒本身就是「记录存在但没被检索到」的证据。按这篇的分析，这不是提醒写得不够醒目的问题，是「按相关性检索」这个形状答不了「这条约束现在还管不管我」这个问题。

但我不打算据此建知识图谱，理由是这篇论文自己给的。它明说：能力优势依赖于正确录入，平铺式录入会让图退化到自由文本同等，而那份纪律的摩擦是向量存储没有的。我这套流水线是每天自动生成的，录入纪律靠的是我当场判断该写进哪一节——这正是「平铺录入」的形状。上一层结构而不解决录入，会得到一个更贵的自由文本。

真正可以低成本抄过来的是那条「取代」边。具体做法：当一条新判断推翻了旧判断时，在新文件里显式写出被推翻的那一条在哪个文件里（而不是只写新结论）。这不需要图数据库，只需要一条约定。它能解掉的是这张表里最疼的那一格——「旧文件和新文件平等」。

第三处值得记的是那个实况试验的设计，因为它把「怎么证明一个记忆系统有用」这件事想清楚了。

问题的难点在于：「结构化记忆能减少数月真实工作里的理解债」这个主张，一个时间点上的基准测不出来。他们的做法：

- 在两个生产代码库上日常使用；
- 试验开始时确立 16 个固定参照点，每个针对一次推翻，且较早的那个做法已从当前代码树里删除——所以答案无法从代码本身重建；
- 标准答案从一手来源撰写，绝不从图里来，因此图不能给自己打分；
- 一个盲的本地判分器按月给「有图 vs 冷启动」的差距打分，对照一个记录下来的第零月基线；
- 预设失败条件：每个项目每月引用记录的协助少于四次，系统就退役。

这里有四个动作值得单拎出来。

第一，参照点要选那些「答案已经不在代码里」的。如果答案还能从当前代码看出来，那这个问题测的是代码可读性、不是记忆系统。

第二，标准答案不能从被测系统里来。这一句是常识，但实际做起来极容易违反——从图里导出问题和答案最省事。

第三，预设失败条件，而且是可执行的（少于四次就退役）。这是在有结果之前先把「什么算失败」定死，避免事后给不好的数字找解释。

第四，报出来的是月度趋势，不是单点。配套地他们诚实报了头 21 天的数字：71 次引用相关记录的主动协助，同时记下 38 次未命中，而且指出 38 次里有 35 次集中在一个项目——那个项目的 commit 历史较薄，引导出来的图也薄。把失败集中在哪里说出来，比只报总数有用得多。

最后说这篇该保持怀疑的地方。

第一，它是一篇 5 页的会议短论文，而且是自家产品的案例研究。作者来自 Trivyn，MOOSE 引擎是专有的（论文原话 proprietary），本体是在他们自家的本体工程工作台里生成的。基准装置与公开语料的运行转录开源了，但私有语料的转录被保留、只给了聚合数字。这不构成不可信，但意味着「自家系统赢」这个结论的独立复现门槛不低。

第二，只评测了一个 agent 家族（GPT-5.x），判分器是 GPT-5.4-mini。论文自己把这一条列进了采纳障碍。而且它指出 MCP 是被动的、依赖 agent 自己决定何时调用，这偏向更高档的模型——所以「小模型受益最大」这个结论（写在结论里）与「依赖高档模型决定调用」这个限制之间是有张力的，论文没有解决这个张力。

第三，那个「优势不随规模复合」的发现，作者报了但没深挖。斜率置信区间跨过零，意味着样本不足以判断斜率的符号——这既可能是真的没有复合效应，也可能是 50 到 634 条这个范围太窄。作者的处理是承认预期落空，这很诚实，但这一条还不是定论。

一句话版本：这篇论文的主张不是「知识图谱比向量检索准」——它主动否掉了那个卖点（检索精度只有恒定小幅领先，斜率不显著）；它的主张是「有三类问题 top-k 检索按形状就答不了：有多少个、有没有、顺着链子走到哪」，而这三类问题恰好是项目记忆最需要回答的。

文中金句

"Source code is the manifestation of what was decided; the reasoning, theory, and rationale behind it live elsewhere." 源代码是「决定了什么」的显现；背后的推理、理论与依据活在别处。（引 Naur 1985）

"Those distinctions are fundamentally ontological. A vector search can find nearby words; it does not know what a record is or what lifecycle role it plays. At that point project memory stops being a note-keeping problem and becomes a modeling problem." 这些区分本质上是本体论的。向量搜索能找到词义相近的东西；它不知道一条记录「是什么」，也不知道它扮演什么生命周期角色。到这一步，项目记忆就不再是一个记笔记的问题，而变成了一个建模问题。

"The underlying neurosymbolic engine treats the LLM as an unreliable, albeit useful sensor." 底层的神经符号引擎把大模型当成一个不可靠、尽管有用的传感器。

"It is not caused by incomplete ingestion: B1-mem0 contains the relevant facts, but its retrieval returns a limited ranked slice, so it cannot enumerate a complete set, establish an absence, or traverse relationships." 这不是摄入不完整造成的：向量基线里含有相关事实，但它的检索返回的是一个有限的排序切片，所以它无法枚举完整集合、无法确立不存在、无法遍历关系。

"The baseline systems, by contrast, could not produce the complete answer at any cost." 相比之下，基线系统在任何代价下都产不出完整答案。

"We had expected the structured advantage to compound as the store grew, but it did not." 我们原本预期结构化的优势会随存储增长而复合，但它没有。

"Gold-standard answers were authored from primary sources, never from the graph, so the graph cannot grade itself." 标准答案从一手来源撰写，**绝不从图里来，因此图不能给自己打分。**

"flat capture degrades the graph to free-text parity, and that discipline imposes friction that a vector store does not." **平铺式的录入会把图退化到与自由文本同等**，而那份纪律带来的摩擦是向量存储没有的。

"Each time a plausible-but-wrong conclusion almost stuck, an auditable, re-checkable process caught it: exactly the failure mode (confident, fluent, wrong) this class of system exists to reduce." 每一次一个**看起来合理但错误**的结论差点站住，都是一个**可审计、可重查的流程**把它抓住了——而这正是这类系统存在的目的所要减少的那种失败模式：**自信、流畅、错误。**

全文翻译

摘要

Coding agents have become the primary means of generating new code in many software projects, and the resulting velocity of changes makes keeping track of the reasons behind those changes challenging.

写代码的 agent 已经成为许多软件项目里产出新代码的主要方式，**而由此带来的改动速度，让追踪这些改动背后的原因变得困难。**

This paper introduces MOOSEDev, a system designed to give coding agents structured, ontology-grounded project memory. The system captures architectural decisions, lessons, constraints, and rationales in a knowledge graph exposed to agents via a Model Context Protocol (MCP) interface.

本文介绍 **MOOSEDev**，一个为写代码的 agent 提供**结构化、以本体为根基的项目记忆**的系统。它把架构决策、教训、约束与依据捕获到一个知识图谱里，通过 MCP 接口暴露给 agent。

Records carry lifecycle status, provenance, and supersession links, queryable via MOOSE, a proprietary neurosymbolic engine that treats the symbolic layer as the primary reasoning substrate.

记录携带**生命周期状态、来源、以及取代链接**，可通过 MOOSE 查询——那是一个专有的神经符号引擎，**它把符号层当作主要的推理基底。**

We compared MOOSEDev against a production vector-memory tool on a neutral public corpus of 835 typed records. MOOSEDev returned the expected answer set essentially in full (0.98 - 1.00) on supersession, set-completeness, and negation questions, whereas the baseline's top-k retrieval surfaced between 6% and 27%.

我们在一个中立的公开语料（835 条有类型的记录）上，把 MOOSEDev 与一个生产级向量记忆工具做了对比。在取代、集合完备性、否定这三类问题上，MOOSEDev 基本完整地返回了预期答案集（0.98 到 1.00），而基线的 top-k 检索只浮出 6% 到 27%。

Conversely, relevance recall and token cost were largely equivalent between the two systems.

反过来，两个系统在相关性召回与 token 成本上大体相当。

我们还描述了对自家代码库做的一次时序化 commit 历史引导、一次预注册的实况试验，以及学到的教训。

一、引言

Software teams increasingly use coding agents for the majority of new code, while keeping architecture and design under the purview of human engineers; teams working this way must continually remind their agents of design decisions and intervene when they go off track.

软件团队越来越多地把大部分新代码交给 agent，而把架构与设计留在人类工程师的职权范围内；这样工作的团队必须不断地向 agent 重申设计决策，并在它们跑偏时介入。

The deeper cost is being called "comprehension debt," the gradual loss of understanding of why a codebase is the way it is.

更深层的代价被称为「理解债」——对「一个代码库为什么长成现在这样」的理解在逐渐流失。

Source code is the manifestation of what was decided; the reasoning, theory, and rationale behind it live elsewhere.

源代码是「决定了什么」的显现；背后的推理、理论与依据活在别处。

Notes files, Markdown specifications, and retrieval-augmented memory help, but persistent issues remain. Agents need to know what kind of knowledge a note represents (decision, constraint, rationale, lesson, anti-pattern); whether it is current or historical; and how records relate: this decision supersedes that one.

笔记文件、Markdown 规格、检索增强记忆都有帮助，但有些问题一直在。agent 需要知道：一条笔记代表哪一种知识（决策、约束、依据、教训、反模式）；它是当前的还是历史的；以及记录之间如何关联——这条决策取代了那条。

Those distinctions are fundamentally ontological. A vector search can find nearby words; it does not know what a record is or what lifecycle role it plays. At that point project memory stops

being a note-keeping problem and becomes a modeling problem.

这些区分本质上是本体论的。 向量搜索能找到词义相近的东西；它不知道一条记录是什么，也不知道它扮演什么生命周期角色。到这一步，项目记忆就不再是一个记笔记的问题，而变成了一个建模问题。

Knowledge graphs over source code itself are well studied ... While those solutions model the code, MOOSEDev models the knowledge about it, a layer that work leaves implicit.

针对源代码本身的知识图谱已经研究得很充分（抽取类与用法的图、用于驱动代码生成的 API 演化图）。**那些方案建模的是代码，而 MOOSEDev 建模的是关于代码的知识**——那些工作留在隐含层里的东西。

本文是一个案例研究：**把写代码的 agent 的记忆当成一个本体论问题来处理。**

二、系统：跑在神经符号引擎上的有类型记录

MOOSEDev 把架构决策、教训、约束、依据、反模式记录在一个项目知识图谱里，**以两个小本体为根基**：software-engineering 提供代码库的结构词汇，software-architecture 提供**关于那个结构的知识**。

Both are OWL ontologies with companion SHACL shapes, generated interactively in Trivyn, our ontology-engineering workbench, refined manually, and reasoned over at runtime by the MOOSE engine; both are deliberately small, nine and eleven classes respectively, 51 properties in total.

两者都是 OWL 本体，各配一套 SHACL 形状约束，在我们的本体工程工作台 Trivyn 里交互式生成、人工精炼，运行时由 MOOSE 引擎做推理；**两者都是刻意做小的——分别 9 个类与 11 个类，共 51 个属性。**

The relationships are crucial: a record can carry a rationale, supersede another record, affect a component, and bear lifecycle status, author, and timestamp; these edges form a traversable network.

关系是关键：一条记录可以携带一个依据、**取代另一条记录**、影响某个组件，并带有生命周期状态、作者与时间戳；**这些边构成一张可遍历的网。**

Structure provides the agent with typed capture, validation against SHACL shapes, queryability (e.g. "accepted decisions with no recorded rationale"), lifecycle and provenance tracking, and the ability to align new concepts.

结构给 agent 带来：有类型的录入、**对照 SHACL 形状做校验**、可查询性（例如「已接受但没有记录依据的决策」）、生命周期与来源追踪，以及对齐新概念的能力。

The underlying neurosymbolic engine treats the LLM as an unreliable, albeit useful sensor. Keyword and structural matching, ontology traversal, deterministic evidence fusion, validation, and execution traces are symbolic; the model is consulted only at narrow, declared points, so it can be small (8 - 32B).

底层的神经符号引擎**把大模型当成一个不可靠、尽管有用的传感器**。关键词与结构匹配、本体遍历、**确定性的证据融合**、校验、执行轨迹，这些都是符号的；**模型只在窄的、声明过的点上被咨询**，因此它可以很小（80 亿到 320 亿参数）。

When an agent asks for current guidance, the engine traverses the graph to find relevant records, follows typed relationships, filters superseded records, and deterministically ranks the remaining evidence into context; the LLM only interprets the question and synthesizes the answer, and every step is logged in an execution trace.

当 agent 请求当前的指导时，引擎遍历图找出相关记录、顺着有类型的关系走、**过滤掉被取代的记录**、把剩余证据确定性地排序进上下文；**大模型只负责理解问题与综合答案，而每一步都被记进一条执行轨迹**。

对外通过 MCP 暴露四组主要工具：有类型的录入、读取（上下文检索、自然语言查询、SPARQL）、生命周期、完整性。

三、评测：结构化记忆对向量记忆

测试设置。

Does an ontology-grounded knowledge graph justify its complexity against free-text and vector memory for a coding agent?

对一个写代码的 agent 来说，一个以本体为根基的知识图谱，相对自由文本与向量记忆，能否证明它那份复杂度的正当性？

基准比较五种记忆与检索条件。**BO** 是没有记忆的 agent（即「vibe coding 模式」）——**地板**。**B1-notes** 把真实文档作为平铺文件提供给 agent。**B1-mem0** 是 mem0（一个当代向量记忆工具），用它自己的原生捕获方法摄入同一批文档。**B1-rag** 是在**图自身内容压平成文本**之后做的基础 BM25 检索，**用来检验内容本身、在没有结构的情况下能否带来同样的结果**。**B2** 是我们的结构化项目记忆图。

The test corpus is the documentation of CodeGraph, a third-party open-source developer tool, captured as 835 typed records in our graph and 553 mem0 memories, verified faithful.

测试语料是 CodeGraph（一个第三方开源开发工具）的文档，在我们的图里录成 **835 条有类型的记录**，在 mem0 里是 **553 条记忆**，**都核对过忠实性**。

A single CLI agent configuration and model (GPT-5.x family) was used for every run; we pre-registered the verdicts, authored ground truth from primary sources or SPARQL-derived sets, and graded with a strict LLM judge (GPT-5.4-mini) that allowed paraphrase but rejected answers on the correct topic with the wrong facts, validated by reproducing the graph condition's strict-match score.

每一次运行都用同一个 CLI agent 配置与同一个模型（GPT-5.x 系列）；**我们预注册了判定**，标准答案来自一手来源或 SPARQL 导出的集合，用一个严格的大模型判分器（GPT-5.4-mini）评分——**它容许改述，但**

拒绝那些主题正确而事实错误的答案，并且通过复现图条件的严格匹配分数来验证这个判分器。

Every run is an immutable transcript, re-gradable without rerunning the agent.

每一次运行都是一份不可变的转录，可以在不重跑 agent 的情况下重新评分。

（结果表见前文「核心内容」一节。）

能力差异。

The results show a qualitative difference on tasks that require structural reasoning.

结果显示，在需要结构化推理的任务上存在性质上的差异。

It is not caused by incomplete ingestion: B1-mem0 contains the relevant facts, but its retrieval returns a limited ranked slice, so it cannot enumerate a complete set, establish an absence, or traverse relationships.

这不是摄入不完整造成的：B1-mem0 里含有相关事实，但它的检索返回的是一个有限的排序切片，所以它无法枚举完整集合、无法确立一个不存在、无法遍历关系。

The structured advantage increases with the size and complexity of the required answer (for a two-item set the systems are equivalent), consistently across corpora.

结构化的优势随所需答案的规模与复杂度增长（两项的集合上两系统等价），且跨语料一致。对 B1-rag 的集合重叠 F1：私有语料 0.94 比 0.25，中立公开语料 0.90 比 0.34。

The capability does cost tokens, however: 78k to 167k for a 122-item completeness set via SPARQL, compared to roughly 35k for targeted retrieval. The baseline systems, by contrast, could not produce the complete answer at any cost.

这个能力确实要花 token：一个 122 项的完备性集合走 SPARQL 要 7.8 万到 16.7 万，而定向检索大约 3.5 万。相比之下，基线系统在任何代价下都产不出完整答案。

This demonstrates the idea that memory is a modeling problem: where a question is structural (e.g., completeness, absence, traversal), the advantage over basic retrieval is categorical.

这印证了「记忆是一个建模问题」这个想法：当一个问题是有结构性的（完备性、不存在、遍历），相对基础检索的优势是范畴性的。

检索上打平，时效靠构造。

Testing only tasks where structured retrieval is required would be a bit of a strawman, so the matrix also tests where vector memory is expected to excel, and there the conditions tie.

只测那些需要结构化检索的任务多少有点稻草人，所以这个矩阵也测了向量记忆本该擅长的地方——而在那里两个条件打平。

简单相关性上，图达到覆盖率 0.82（约 3.5 万 agent token），mem0 在 0.67 到 0.90 之间（约 4 万）。

We had expected the structured advantage to compound as the store grew, but it did not: a scale study from 50 to 634 records found the graph ahead at every size (hit@5 0.84 versus 0.60 at the largest) at a constant offset; the slope's confidence interval spans zero.

我们原本预期结构化优势会随存储增长而复合，但它没有：一个从 50 条到 634 条的规模研究发现图在每个规模上都领先（最大规模 hit@5 是 0.84 比 0.60），**但是一个恒定的偏移量；斜率的置信区间跨过零。**

Retrieval precision is thus not the differentiator, and on its own it would likely not justify the system. Currency is different.

因此检索精度不是区分点，单靠它大概不足以证明这个系统的正当性。时效则不同。

across four real-world reversal pairs, four models, and two delivery regimes, the graph served the current answer in all 40 trials, since superseded records are excluded from current-guidance retrieval by construction.

跨四对真实的推翻关系、四个模型、两种投递方式，**图在全部 40 次试验中都给出了当前的答案，因为被取代的记录按构造就被排除在当前指导的检索之外。**

在被测的那次推翻上，B1-mem0 与图打平、都是 100%（而冷启动的 agent 是 0.00）；**而自由文本那一条，在被取代记录排名高于当前记录、且该内容被推进提示词的那一对上，13 次试验里只有 1 次是当前的（8%）。**

四、部署、引导与教训

给已有仓库做引导。

A graph built from only the current state of a repository would be flat, missing the historical relationships, so our bootstrap workflow walks repository history commit by commit, extracting typed records with historical timestamps. Over our own repository, it recovered seven real supersession chains.

只用仓库当前状态建出来的图会是平的、缺少历史关系，所以我们的引导流程**逐个 commit 走过仓库历史，抽取带历史时间戳的有类型记录。**在我们自己的仓库上，它**恢复出七条真实的取代链。**

The ontologies are domain-level rather than project-specific, so a new codebase needs no custom schema work, only the instance graph built during bootstrap.

这些本体是领域级的、不是项目专用的，所以一个新代码库不需要任何定制 schema 工作，只需要引导过程中建出的实例图。

一次预注册的实况试验。

The claim that motivated this project, that structured memory reduces comprehension debt over months of real work, cannot be tested by a point-in-time benchmark.

驱动这个项目的那个主张——结构化记忆能在数月的真实工作里减少理解债——无法用一个时间点上的基准来检验。

所以他们在公司的两个生产代码库上做日常使用测试。

We established sixteen fixed reference points for context recovery at the start of testing, each targeting a reversal whose earlier approach is deleted from the current tree, so the answer cannot be reconstructed from the code alone.

测试开始时确立了**十六个固定参照点**用于上下文恢复，**每一个都针对一次推翻**，且较早的那个做法已从当前代码树中删除，因此答案无法仅凭代码重建。

Gold-standard answers were authored from primary sources, never from the graph, so the graph cannot grade itself.

标准答案从一手来源撰写，绝不从图里来，因此图不能给自己打分。

A blind local judge scores a monthly graph-versus-cold gap against a recorded month-zero baseline, with a pre-set failure condition: fewer than four record-citing assists per project per month triggers the system's retirement.

一个**盲的**本地判分器按月给「有图对冷启动」的差距打分，对照一个记录下来的第零月基线，并预设了失败条件：**每个项目每月引用记录的协助少于四次，就触发这个系统的退役。**

The registered readout is the monthly trend; in the trial's first twenty-one days the graph provided 71 unprompted assists citing relevant records, while logging 38 misses, 35 of 38 in one project bootstrapped from a thinner commit history.

注册的读数是月度趋势；试验头 21 天里，图提供了 **71 次引用相关记录的、未经提示的协助**，同时记录了 **38 次未命中**（一次无用的召回，或一份它本应携带的知识）——**38 次里有 35 次出在同一个项目**，那个项目是从较薄的 commit 历史引导出来的。

给实践者的教训。

Measuring an LLM-based memory system misleads in predictable ways. Our evaluation nearly shipped three wrong conclusions: a 20-point win for our own system that was a grading artifact; an apparent relevance loss caused by a configuration bug: the memory server was down and agents silently grepped the raw store; and a misleading commit message that poisoned both a gold answer and a bootstrapped rationale.

衡量一个基于大模型的记忆系统，会以可预测的方式误导人。我们的评测差点发出三个错误结论：自家系统赢 20 分——**那是评分的产物**；一个表面上的相关性损失——**由一个配置 bug 造成**：记忆服务器当时是宕的，

而 agent 静默地去 grep 了原始存储；以及一条误导性的 commit message，同时污染了一个标准答案和一条引导出来的依据。

The rules that caught them generalize beyond benchmarks: immutable transcripts with free re-grades, a judge validated against a condition with a known score, runs that verify the memory tool actually fired, and ground truth only from primary sources.

抓住它们的那些规则可以推广到基准之外：可自由重新评分的不可变转录；在一个已知分数的条件上验证过的判分器；核实记忆工具真的被触发了的运行；以及只来自一手来源的标准答案。

Each time a plausible-but-wrong conclusion almost stuck, an auditable, re-checkable process caught it: exactly the failure mode (confident, fluent, wrong) this class of system exists to reduce.

每一次一个看起来合理但错误的结论差点站住，都是一个可审计、可重查的流程把它抓住的——而这正是这类系统存在的目的所要减少的那种失败模式：自信、流畅、错误。

采纳的障碍。

The capability win depends on correct capture: flat capture degrades the graph to free-text parity, and that discipline imposes friction that a vector store does not.

这份能力上的胜出依赖于正确的录入：平铺式录入会把图退化到与自由文本同等，而那份纪律带来的摩擦是向量存储没有的。对齐工具、辅助录入，以及结论的推送方向，都能降低这个成本。

The design also leans on host tool use: MCP is passive; the agent decides when to call, which favors higher-tier models; we evaluated one agent family.

这个设计还倚赖宿主的工具使用：MCP 是被动的，由 agent 决定何时调用，这偏向更高档的模型；我们只评测了一个 agent 家族。

五、结论

Against a common vector-memory tool, ontology-grounded memory (given effective capture) ties where retrieval is strong, and is clearly better on completeness, absence, and supersession.

相对一个常见的向量记忆工具，以本体为根基的记忆（在录入有效的前提下）在检索占优的地方打平，而在完备性、不存在、取代这三项上明显更好。

Small local models with limited context windows should benefit most, so the natural next step is to act inside the loop: pushing entity-exact records when code is touched and gating edits on recorded constraints.

上下文窗口有限的小型本地模型应当受益最大，所以自然的下一步是在循环内部行动：代码被触碰时推送实体精确的记录，并用已记录的约束给编辑加闸。

参考链接

- 论文 [arXiv 2608.13662](#) — NeSy 2026, 5 页
- MOOSEDev 基准装置与公开语料转录 — [bench/](#) 目录；私有语料转录未公开，仓库里只有聚合结果
- Naur, [Programming as Theory Building \(1985\)](#) — 「源代码只是决定的显现」这个判断的出处
- Nygard, [Documenting Architecture Decisions \(2011\)](#) — 架构决策记录（ADR）的原始博文，这篇的记录类型直接沿用它
- [mem0](#) — 被当作向量记忆基线的那个工具
- [SHACL](#) — 形状约束语言，这篇用它给记录做校验
- [Model Context Protocol](#) — 系统对外暴露工具的接口